

AUTORIZACIÓN FRONTEND

Clase #20 - 14 de septiembre de 2026

Matías Corti - Martina García Améndola



¡CERRARON EL BACKEND!

Ayer entregaron una API real:

- ✓ Express + TypeScript con estructura por capas (rutas, controllers, services)
- ✓ PostgreSQL en Docker + Prisma, con relaciones y migraciones commiteadas
- ✓ Validación con Zod y manejo de errores centralizado
- ✓ Registro, login, authenticate y authorize según su matriz de permisos
- ✓ Un frontend que ya interactúa con ese backend, con el token viajando
- ✓ Trabajo en equipo, con Git y PRs

En julio muchos nunca habían levantado un servidor. Tres entregas, tres capas: front, back, y la integración entre las dos. No es poco!



DÓNDE QUEDAMOS

La clase pasada cerramos con esta deuda. Qué pasa con ella hoy:

Pendiente de C19	Lo resuelve
El token vive en localStorage y se lee con dos funciones sueltas	← HOY · AuthContext
No hay AuthContext ni rutas protegidas en React	← HOY · AuthProvider + PrivateRoute
Los botones se ven aunque no puedas usarlos	← HOY · renderizado por rol
La matriz de permisos vive solo en el back	← HOY · la misma matriz, columna UI
HTTPS y CORS con dominio real	Clase de Deploy
Testing de la integración	Clase de Testing

TEMAS DE HOY

- Autenticación vs autorización, ahora desde la UI
 - Por qué la autorización en el front no es seguridad
 - `AuthContext`: la sesión como estado global (el patrón de C12, otro dato)
 - Recargar la página: rehidratación y el estado `cargando`
 - Rutas protegidas: `PrivateRoute`, `Outlet`, `Navigate`
 - Renderizado condicional por rol
 - Qué pasa cuando vence el token en medio de la sesión
 - Dónde guardar el token: `localStorage` vs `cookie httpOnly`
 - Práctica: cinco pasos, la app anda en cada uno
 - Tres piezas que hoy se juntan: `context` (C12), el rol en el token (C18) y el token en front (C19)
-



LA IDEA DE HOY

El backend **ya decide** quién puede hacer qué: 401 y 403. Eso no cambia.

Hoy hacemos que la UI **sepa quién sos**, para no mostrarte puertas que no podés abrir.

La autorización en el front es **UX y defensa en profundidad. No es seguridad.** La seguridad sigue en el back.

Es la misma regla de siempre, tercera vez:

Clase	Regla
C10	Validar en el front es UX, no seguridad
C19	Si te importa que nadie lo lea, no puede estar en el front
C20	Esconder una puerta en el front es UX, no seguridad



AUTENTICACIÓN VS AUTORIZACIÓN, DESDE EL FRONT

Las dos preguntas de C18 no cambian. Cambia **quién reacciona.**

	 Autenticación	 Autorización
Pregunta	¿Quién sos?	¿Qué podés hacer?
Quién responde	El back (authenticate)	El back (authorize)
Si falla	401	403
Lo que sabe la UI	Si hay un usuario en el context	El rol de ese usuario
Lo que hace la UI	Muestra login o "Hola, {nombre}"	Muestra u oculta puertas
Cuándo se equivoca la UI	El back lo corrige con 401	El back lo corrige con 403



401 VS 403: QUÉ HACE LA UI

En C18 fueron dos status. Hoy son dos **destinos**.

Status	El back dice	La UI hace	Ruta
401	"No sé quién sos"	Cierra la sesión (si había) y manda a loguearse	/login
403	"Sé quién sos, y no podés"	Avisa que no alcanza el permiso. No cierra la sesión	/sin-permiso

Y la UI puede **anticiparlos** sin llamar al back:

- Sin **usuario** en el context → ni intentes: **/login**
- Con **usuario** pero sin el rol → ni intentes: **/sin-permiso**



LA UI NO ES SEGURIDAD

¿Qué es "defensa en profundidad"?

- Dos capas que dicen lo mismo: la de afuera (UI) y la de adentro (API)
- La de afuera **mejora la experiencia**: no ves lo que no podés usar
- La de adentro **es la regla**: aunque saltes la UI, la API dice no

¿Por qué el front no puede ser la regla?

- Todo el JS del front está en el navegador del usuario. Lo puede leer y editar
- Escribir la URL a mano saltea cualquier botón escondido
- `curl` ni siquiera pasa por React

La prueba: borra el `if` que esconde el botón, hacé el POST como CLIENTE → **403 igual.**



LA MATRIZ DE C18, CON UNA COLUMNA NUEVA

Cada fila tiene **dos implementaciones**: la que vale (back) y la que se ve (UI).

Endpoint	Invitado	CLIENTE	ADMIN	Qué hace la UI
GET /api/libros /api/libros/:id	✓	✓	✓	Catálogo público. Sin login
POST /api/libros	401	403	201	/libros/nuevo detrás de PrivateRoute rol="ADMIN". Link "Nuevo libro" solo ADMIN
PUT DELETE /api/libros/:id	401	403	2xx	Botones editar/borrar solo ADMIN (no existen todavía: extensión / TP)
GET /api/auth/yo	401	✓	✓	La usa el AuthProvider al recargar

Para el TP: la matriz de permisos que ya está en [Documentación/](#) gana esta columna. Es parte de [\[AG-C20\]](#).



MINI-CIERRE: AUTORIZACIÓN DESDE LA UI

Concepto	En una línea
Autenticación / autorización	¿Quién sos? / ¿Qué podés? El back responde 401 / 403. La UI reacciona
La UI anticipa	Sin usuario → /login. Sin rol → /sin-permiso. Sin llamar al back
Defensa en profundidad	Dos capas que dicen lo mismo. La de afuera no reemplaza la de adentro
La UI no es seguridad	Todo el front es editable por el usuario. La regla vive en la API
Matriz + columna UI	Cada permiso: una implementación que vale y una que se ve



EL HEADER QUE FUNCIONA DE CASUALIDAD

Así quedó `Header.tsx` en C19:

```
// frontend/src/components/Layout/Header.tsx (C19)
const estaLogueado = !!obtenerToken(); // lee localStorage en cada render
...
{estaLogueado ? 'Salir' : 'Ingresar'}
```

Anda. Pero ¿por qué anda?

- `localStorage` no es estado de React: cambiarlo **no re-renderiza nada**
- Después del login el botón dice "Salir" solo porque `navigate('/catalogo')` re-renderiza el Header
- Sacá el `navigate` y el botón se queda en "Ingresar" con el token guardado
- Y el Header no sabe **quién** sos: solo si hay un string en `localStorage`

La sesión es estado. Estado transversal (Header, páginas, rutas). Eso es un **context**.



AUTHCONTEXT: EL PATRÓN DE C12 CON OTRO DATO

Mismas tres piezas. Cambia el dato que reparte.

Pieza	C12 • BusquedaContext	C20 • AuthContext
1. El contexto	<code>createContext<BusquedaContextType null>(null)</code>	<code>createContext<AuthContextType null>(null)</code>
2. El proveedor	BusquedaProvider: tiene filtro	AuthProvider: tiene usuario y cargando
3. El hook propio	<code>useBusqueda()</code> tira si falta el Provider	<code>useAuth()</code> tira si falta el Provider
El dato	Un string de búsqueda	Quién está logueado y qué puede hacer
Quién lo consume	Header y Catálogo	Header, Login, PrivateRoute, cualquier página

En C12 dijimos qué **sí** va a un context: filtro, tema, usuario logueado.



QUÉ GUARDA EL CONTEXT

```
// frontend/src/context/AuthContext.tsx
interface AuthContextType {
  usuario: Usuario | null;           // null = nadie logueado
  cargando: boolean;                // true mientras averiguamos quién sos
  estaAutenticado: boolean;         // usuario !== null, para leer más cómodo
  tieneRol: (rol: Rol) => boolean;   // usuario?.rol === rol
  login: (credenciales: Credenciales) => Promise<void>;
  logout: () => void;
}
```

- `usuario` es el mismo objeto que devuelve el back:
`{ id, email, nombre, rol }`
- `login` hace el `POST /auth/login` **adentro** del Provider: la página ya no toca el token
- `logout` borra el token y pone `usuario` en `null`
- El **token no está** en el context. Lo maneja `sesion.ts`, y solo lo leen `apiFetch` y el Provider



REHIDRATACIÓN: RECARGÁS LA PÁGINA Y... ¿QUIÉN SOS?

El problema: F5 borra todo el estado de React. El token sigue en localStorage. `usuario` arranca en `null`.

Dos formas de recuperar la sesión:

	A • Decodificar el token en el front	B • Preguntarle al back
Cómo	<code>atob(token.split('.')[1]) → { id, rol, exp }</code>	GET /auth/yo con el token
Sincrónico	Sí	No: hay un rato de "no sé"
Fuente de verdad	El token, tal como está en el navegador	El back: valida firma y vencimiento
Si el token está vencido	La UI cree que estás logueado hasta que un request falla	401 → limpiás y listo
Datos del usuario	Solo lo que hay en el payload (id, rol). El nombre no	Todo { id, email, nombre, rol }
Código	Decode <code>base64url</code> , parse, comparar exp	Un <code>apiFetch</code>

Decisión del curso: B. Reutiliza el endpoint que construimos en C18 para esto. El back es la fuente de verdad.



EL ESTADO CARGANDO

```
// frontend/src/context/AuthContext.tsx
const [usuario, setUsuario] = useState<Usuario | null>(null);
const [cargando, setCargando] = useState(obtenerToken() !== null);

useEffect(() => {
  if (!obtenerToken()) return; // sin token no hay nada que averiguar
  apiFetch<Usuario>('/auth/yo')
    .then(setUsuario)
    .catch(() => borrarToken()) // vencido o inválido: se limpia
    .finally(() => setCargando(false));
}, []);
```

La sesión tiene **tres** estados, no dos:

cargando	usuario	Significa	La UI
true	null	Hay token, todavía no sé quién sos	Spinner. No decidas nada
false	null	Nadie logueado	"Ingresar"
false	{...}	Logueado	"Hola, {nombre}"



DÓNDE VA EL PROVIDER

```
// frontend/src/App.tsx
<AuthProvider>
  <BusquedaProvider>
    <Layout>
      <Routes> ... </Routes>
    </Layout>
  </BusquedaProvider>
</AuthProvider>
```

- Misma regla que en C12: **por encima de todo lo que lo consume**
- El Header (dentro de `Layout`) lo consume → el Provider va arriba de `Layout`
- `AuthProvider` no usa hooks del router → puede ir arriba o abajo del `BrowserRouter`. Lo dejamos en `App.tsx`, al lado del otro
- Dos providers anidados es normal. Diez, no: ahí se piensa otra cosa



DÓNDE GUARDAR EL TOKEN: LOCALSTORAGE VS COOKIE HTTPONLY

	localStorage (lo que hacemos)	Cookie httpOnly
Quién manda el token	apiFetch, en el header Authorization	El navegador, solo, en cada request
¿El JS lo puede leer?	Sí	No
Riesgo principal	XSS: un script inyectado lee el token	CSRF: el navegador lo manda sin preguntar → SameSite, token anti-CSRF
CORS	origin alcanza	+ credentials: true de los dos lados. origin no puede ser comodín
Cambios en el back	Ninguno	Set-Cookie en login, cookie-parser, leer de req.cookies, endpoint de logout
Logout	Borrar la clave	Pedirle al back que borre la cookie
Complejidad	Baja	Media: toca CORS, back y front

Decisión de la materia: seguimos con localStorage, con el riesgo declarado.

Mitigaciones que ya tenemos: React escapa el HTML por defecto, token corto (2h), nada sensible en el payload.

En producción web, la recomendación general es cookie `httpOnly` + `SameSite`.



MINI-CIERRE: AUTHCONTEXT

Concepto	En una línea
Por qué un context	La sesión es estado transversal. localStorage no re-renderiza
Qué guarda	usuario, cargando, login, logout, estaAutenticado, tieneRol. El token no
Rehidratación	F5 borra el estado, no el token. Se le pregunta al back (/auth/yo)
cargando	El tercer estado: "todavía no sé". Sin él, la UI decide con datos viejos
El Provider	Arriba de todo lo que lo consume, como en C12
localStorage vs httpOnly	XSS vs CSRF. La materia: localStorage con riesgo declarado



RUTAS PROTEGIDAS: LA LAYOUT ROUTE

¿Qué es? En react-router, un `<Route>` **sin path** que envuelve a otras rutas. Su `element` se renderiza siempre que matchee una ruta hija, y decide dónde va la hija con `<Outlet />`.

```
// frontend/src/App.tsx
<Route path="/" element={<Home />} />
<Route path="/login" element={<Login />} />
<Route path="/sin-permiso" element={<SinPermiso />} />

{/* sin path: layout route */}
<Route element={<PrivateRoute rol="ADMIN" />}>
  {/* se renderiza en el <Outlet /> de PrivateRoute */}
  <Route path="/libros/nuevo" element={<LibroNuevo />} />
</Route>
```

¿Por qué así y no `<PrivateRoute><LibroNuevo /></PrivateRoute>` en cada ruta?

- La protección se declara **una vez** y agrupa: diez rutas de admin, un solo `<Route>` padre
- Se lee en `App.tsx` de un vistazo qué es público y qué no
- Es la forma idiomática de react-router v7



PRIVATEROUTE: TRES PREGUNTAS, EN ORDEN

```
// frontend/src/components/PrivateRoute.tsx
function PrivateRoute({ rol }: { rol?: Rol }) {
  const { usuario, cargando } = useAuth();

  // 1. ¿ya sé quién sos?
  if (cargando) return <Spinner animation="border" />;
  // 2. ¿sos alguien? (401)
  if (!usuario) return <Navigate to="/login" replace />;
  // 3. ¿podés? (403)
  if (rol && usuario.rol !== rol) return <Navigate to="/sin-permiso" replace />;

  // sí: pasá
  return <Outlet />;
}
```

Uso	Exige
<code><Route element={<PrivateRoute />>></code>	Estar logueado (cualquier rol)
<code><Route element={<PrivateRoute rol="ADMIN" />>></code>	Estar logueado y ser ADMIN

El orden importa: no podés preguntar el rol de alguien que todavía no sabés quién es. Es `authenticate` antes de `authorize`, en el front.



DOS BUGS QUE NO SE VEN HASTA QUE SE VEN

Bug	Qué hacés	Qué pasa	Por qué
El usuario atrapado	<code><Navigate to="/login" > sin replace</code>	Llegás a /login, tocás "atrás", volvés a /login	Sin replace, /libros/nuevo queda en el historial. "Atrás" vuelve ahí y te redirige de nuevo
El flash a /login	Sacás el if (cargando)	Logueado, F5 en /libros/nuevo → te tira a /login	En el primer render usuario es null (todavía no llegó /auth/yo). PrivateRoute decide con datos viejos

- **replace** **reemplaza** la entrada del historial en vez de agregar una. La redirección no deja rastro
- **cargando** le dice a **PrivateRoute** "esperá". Sin el tercer estado, no hay forma de esperar



RENDERIZADO CONDICIONAL POR ROL

Es JSX de C08: `{condición && <Elemento />}`. Lo nuevo es de dónde sale la condición.

```
// frontend/src/components/Layout/Header.tsx
const { usuario, logout, tieneRol } = useAuth();
...
{tieneRol('ADMIN') && <Nav.Link as={NavLink} to="/libros/nuevo">Nuevo
libro</Nav.Link>}
{usuario && <Navbar.Text>Hola, {usuario.nombre}</Navbar.Text>}
```

Qué	Quién lo ve	Cómo
"Ingresar"	Nadie logueado	!usuario
"Hola, {nombre} · Salir"	Cualquier logueado	usuario
Link "Nuevo libro"	Solo ADMIN	tieneRol('ADMIN')
Botones editar / borrar en la card (TP)	Solo ADMIN	tieneRol('ADMIN')

Esconder ≠ proteger. Escondés el link y protegés la ruta. Las dos. Si escribís la URL a mano, `PrivateRoute` te para. Si saltás las dos, el back te para.



EL TOKEN VENCE EN MEDIO DE LA SESIÓN

Estás logueado hace 2 horas y 1 minuto. Tocás "Agregar libro".

Paso	Quién	Qué
1	apiFetch	POST /libros con el token
2	Back (authenticate)	401 { error: "Token expirado" }
3	apiFetch	Ve 401 y que mandó token → dispara <code>window.dispatchEvent(new Event('sesion-expirada'))</code> . Tira <code>ApiError</code>
4	AuthProvider	Escucha el evento → <code>logout()</code>
5	La UI	Navbar: "Ingresar". PrivateRoute (si estabas en una) → /login. El form muestra "Token expirado"

```
// frontend/src/services/api.ts
export class ApiError extends Error { status: number; ... }
...
if (res.status === 401 && token) window.dispatchEvent(new Event('sesion-expirada'));
if (!res.ok) throw new ApiError(res.status, cuerpo?.error ?? `Error ${res.status}`);
```

¿Por qué un evento? `apiFetch` no es un componente: no puede usar `useAuth()`. Avisa al mundo; el que sabe qué hacer, escucha. Cero acoplamiento.

¿Por qué "y mandó token"? El 401 de un login fallido (sin token) **no** es una sesión vencida.



MINI-CIERRE: RUTAS Y UI POR ROL

Concepto	En una línea
Layout route	<code><Route></code> sin path, con <code><Outlet /></code> . Protege un grupo de rutas de una vez
PrivateRoute	cargando → usuario → rol → <code><Outlet /></code> . Mismo orden que <code>authenticate</code> → <code>authorize</code>
Por login vs por rol	Sin usuario → <code>/login</code> . Con usuario sin rol → <code>/sin-permiso</code>
Navigate replace	Sin replace, "atrás" te atrapa en <code>/login</code>
El flash	Sin cargando, <code>PrivateRoute</code> decide con usuario viejo
Renderizado condicional	<code>{tieneRol('ADMIN') && ...}</code> . Esconder y proteger
Token vencido	401 con token → evento → <code>logout()</code> . El back avisa, el front obedece



PRÁCTICA



PRÁCTICA

CINCO PASOS

Cada paso deja la app andando y tiene su commit en el branch **C20**. Si te trabás, `git checkout` al commit del paso anterior y seguís.

#	Paso	Qué cambia	Min
0	Preparar el terreno	Nada: C19 andando	5
1	AuthContext.tsx + Provider	Nuevo archivo. Nadie lo usa todavía	10
2	Login y Header consumen useAuth()	"Hola, Admin". Adiós obtenerToken en componentes	10
3	PrivateRoute + SinPermiso + rutas anidadas	/libros/nuevo solo ADMIN	15
4	Link "Nuevo libro" solo ADMIN	Renderizado condicional	5
5	ApiError + sesion-expirada	El token vence → la sesión se cierra sola	10

Sin `npm install`. Todo lo que se usa ya está instalado.

Ejemplo completo: **DS26-Libreria (C20)**



PASO 0

PREPARAR EL

TERRENO

[Ejemplo Completo:](#)
[DS26-Libreria \(C20\)](#)

Listo cuando: te logueás como `admin@libreria.test / Admin1234` y el botón dice "Salir". Es decir: C19 anda.

```
docker compose ps                # api y db arriba, db "healthy"
docker compose exec api npx prisma generate
cd frontend && npm run dev        # http://localhost:5173
```

⚠ Si te pasa esto	Es porque
El login da Credenciales inválidas con el seed	No corriste el seed, o corriste migrate reset y no lo volviste a correr
Failed to fetch	La API no está arriba. docker compose ps
Tu front de C19 no compila	Plan B: git checkout C19 del ejemplo y trabajá sobre eso
No tenés el /auth/yo en tu back del TP individual	Lo agregás (C18, tres líneas: ruta + authenticate + findById). Hoy se usa



PASO 1

AUTHCONTEXT.TSX

+ PROVIDER

[Ejemplo Completo:](#)
[DS26-Libreria \(C20\)](#)

Archivos a crear / modificar

Archivo	Qué
frontend/src/types/sesionType.ts	Sacar Usuario y Rol a tipos propios. Sesion = { token, usuario: Usuario }. Agregar Credenciales
frontend/src/context/AuthContext.tsx	Nuevo. Copiá BusquedaContext.tsx y cambiá el dato. Filminas 13 y 15
frontend/src/App.tsx	<AuthProvider> envolviendo a <BusquedaProvider>

Listo cuando: la app anda **exactamente igual** que antes, y `npx tsc -p tsconfig.app.json --noEmit` no da errores. Nadie consume el context todavía: ese es el punto.

⚠ Si te pasa esto	Es porque
'ReactNode' is a type and must be imported using a type-only import	verbatimModuleSyntax: import type { ReactNode }
ESLint: Avoid calling setState() directly within an effect	Inicializaste cargando en true y hacés setCargando(false) en el efecto. Inicializalo con obtenerToken() !== null
Property 'nombre' does not exist on type 'Sesion'	login() tiene que guardar sesion.usuario, no sesion
En Network aparecen dos GET /auth/yo al recargar	StrictMode en dev. Es normal (Rompelo #10)

Commit de rescate: `feat(c20): AuthContext + AuthProvider en App`



PASO 2

LOGIN Y HEADER

CONSUMEN

USEAUTH()

[Ejemplo Completo:](#)
[DS26-Libreria \(C20\)](#)

```
// frontend/src/pages/Login.tsx - el onSubmit
const { login } = useAuth();
...
await login(datos);           // el token y el usuario quedan en el AuthProvider
navigate('/catalogo');
```

```
// frontend/src/components/Layout/Header.tsx
const { usuario, logout } = useAuth();

const manejarSesion = () => {
  if (usuario) {
    logout();           // antes: borrarToken()
    navigate('/');
  }
  else navigate('/login');
};
...
{usuario && <Navbar.Text className="ms-lg-3">Hola, {usuario.nombre}</Navbar.Text>}
<button className="btn-login ..." onClick={manejarSesion}>
  {usuario ? 'Salir' : 'Ingresar'}
</button>
```

Archivo	Qué
frontend/src/pages/Login.tsx	Se van apiFetch, guardarToken, Sesion. Entra useAuth
frontend/src/components/Layout/Header.tsx	Se va obtenerToken/borrarToken. Entra useAuth. Navbar.Text con el nombre



PASO 2

LISTO CUANDO Y QUÉ PUEDE FALLAR

[Ejemplo Completo:](#)
[DS26-Libreria \(C20\)](#)

Listo cuando: logueás como admin y la navbar dice **"Hola, Admin • Salir"** sin depender del `navigate`. Recargás (F5) y sigue diciéndolo. "Salir" vuelve a "Ingresar".

⚠ Si te pasa esto	Es porque
La navbar sigue en "Ingresar" después de loguear	Login.tsx sigue llamando a guardarToken directo en vez de login(). El token está, el estado no
Cannot find name 'apiFetch' en Login.tsx	Borraste el import y quedó un uso. Se va todo
Pantalla blanca: useAuth debe usarse dentro de <AuthProvider>	El Provider no envuelve al Header. Mismo error que en C12

Commit de rescate: `feat(c20): Login usa login() del context; Header muestra Hola {nombre} / Salir`



CHECKPOINT

PRÁCTICA

Qué tenés que ver, antes de seguir:

Situación	Qué ves
Sin loguear	Navbar: "Ingresar"
Login con password mal	Credenciales inválidas (el error viene del back, igual que en C19)
Login como admin	Navbar: "Hola, Admin · Salir". Un solo request: POST /auth/login
F5	Spinner imperceptible, GET /auth/yo, sigue "Hola, Admin"
Salir	Token borrado, navbar "Ingresar"
Editás el token en Application → Local Storage y F5	GET /auth/yo → 401 Token inválido. Token borrado. "Ingresar"

Lo que **todavía** pasa: entrás a `/libros/nuevo` sin loguearte y te deja.
Eso es lo que sigue.



PASO 3

PRIVATEROUTE + SINPERMISO + RUTAS ANIDADAS

[Ejemplo Completo:](#)
[DS26-Libreria \(C20\)](#)

Archivos a crear / modificar

Archivo	Qué
frontend/src/components/PrivateRoute.tsx	Nuevo. Filmina 22, tal cual
frontend/src/pages/SinPermiso.tsx	Nuevo. Alert variant="warning" + Link al catálogo. La cara del 403
frontend/src/App.tsx	/sin-permiso pública. /libros/nuevo adentro de <Route element={<PrivateRoute rol="ADMIN" />}>

Listo cuando:

- Sin loguear, escribís `/libros/nuevo` → `/login`, sin ningún request al back
- Como CLIENTE → `/sin-permiso`
- Como ADMIN → el formulario. F5 ahí mismo → spinner brevísimo, seguís en el formulario

⚠ Si te pasa esto	Es porque
<code>/libros/nuevo</code> en blanco como ADMIN, sin errores	Falta el <code><Outlet /></code> en <code>PrivateRoute</code> . La hija no tiene dónde renderizarse
Logueado, F5 y te manda a <code>/login</code>	Falta el <code>if (cargando)</code> (filmina 23)
Spinner eterno	Falta el <code>.finally(() => setCargando(false))</code> , o la API está caída
LibroNuevo se renderiza sin pasar por <code>PrivateRoute</code>	Quedó una segunda <code><Route path="/libros/nuevo"></code> afuera del padre. Borrala

Commit de rescate: `feat(c20): PrivateRoute (layout route + Outlet + Navigate) y página SinPermiso`



PASO 4

LINK "NUEVO LIBRO" SOLO PARA ADMIN

Ejemplo Completo:
DS26-Libreria (C20)

Archivo	Qué
frontend/src/components/Layout/Header.tsx	<code>tieneRol del useAuth(). {tieneRol('ADMIN') && <Nav.Link as={NavLink} to="/libros/nuevo">Nuevo libro</Nav.Link>}</code>

Listo cuando: como CLIENTE no ves el link. Como ADMIN sí. Como CLIENTE escribís la URL a mano → `/sin-permiso` igual (paso 3 sigue vigente).

! Si te pasa esto	Es porque
El link se ve para todos	Pusiste usuario <code>&&</code> en vez de <code>tieneRol('ADMIN')</code> <code>&&</code> . Logueado no es admin
<code>tieneRol</code> is not a function	No la agregaste al value del Provider, o no la desestructuraste del <code>useAuth()</code>

Commit de rescate: `feat(c20): link "Nuevo libro" en la navbar solo para ADMIN`



PASO 5

APIERROR +

SESION-EXPIRADA

[Ejemplo Completo:](#)
[DS26-Libreria \(C20\)](#)

Archivo	Qué
frontend/src/services/api.ts	class ApiError extends Error { status }. Antes del throw: if (res.status === 401 && token) window.dispatchEvent(new Event('sesion-expirada')). El throw pasa a new ApiError(res.status, ...)
frontend/src/context/AuthContext.tsx	Un useEffect más: addEventListener('sesion-expirada', logout) con su removeEventListener en el cleanup

```
// frontend/src/context/AuthContext.tsx
useEffect(() => {
  window.addEventListener('sesion-expirada', logout);
  return () => window.removeEventListener('sesion-expirada', logout);
}, []);
```

Listo cuando: `useFetch` y las páginas **no cambiaron** y todo sigue andando. Para verlo funcionar hace falta un token vencido: Rompelo #9.

⚠ Si te pasa esto	Es porque
Un login con password mal te "desloguea"	Sacaste el <code>&& token</code> de la condición. Un 401 sin token no es sesión vencida
Con el token vencido la navbar no cambia	El nombre del evento no coincide entre <code>api.ts</code> y <code>AuthContext.tsx</code> , o el listener no está en un <code>useEffect</code>

Commit de rescate: `feat(c20): ApiError con status; 401 con token dispara "sesion-expirada"`



PROBÁ LA MATRIZ, AHORA CON LA UI

Situación	RoI	Qué hacés	Qué ves
Sin login	-	Mirás la navbar	"Ingresar". Sin "Nuevo libro"
Sin login	-	/libros/nuevo a mano	→ /login. Network: nada
Logueado	CLIENTE	Mirás la navbar	"Hola, Cliente · Salir". Sin "Nuevo libro"
Logueado	CLIENTE	/libros/nuevo a mano	→ /sin-permiso. Network: nada
Logueado	ADMIN	Mirás la navbar	"Hola, Admin · Nuevo libro · Salir"
Logueado	ADMIN	Nuevo libro → completás → Agregar	POST /libros → 201, vuelve al catálogo
Logueado	cualquiera	F5 en cualquier página	GET /auth/yo → sigue logueado
Logueado	cualquiera	Salir	"Ingresar". Token borrado
Token editado a mano	-	F5	GET /auth/yo → 401 Token inválido. Sesión limpia

Seed: `admin@libreria.test / Admin1234 · cliente@libreria.test / Cliente1234`

Compará con la tabla de C19-31: **el 401 y el 403 ya no llegan al usuario**. Los anticipa la UI. Siguen ahí, en el back, por si alguien saltea la UI.

Ejemplo Completo:
DS26-Libreria (C20)



ROMPELO A PROPÓSITO

Diez fallas con su síntoma real. Elegí tres y reproducilas.

#	Qué hacés	Qué ves
1	Sacá replace del <code><Navigate to="/login" /></code>	Llegás a /login. "Atrás" → /login. Atrapado
2	Sacá el <code><Outlet /></code> de PrivateRoute	/libros/nuevo como ADMIN: página en blanco. Cero errores
3	Usá <code>useAuth()</code> en un componente fuera del Provider	Error: <code>useAuth</code> debe usarse dentro de <code><AuthProvider></code>
4	Sacá el <code>if</code> (cargando)	Logueado, F5 en /libros/nuevo → /login. El flash
5	Application → Local Storage → cambiá un carácter del token → F5	GET /auth/yo 401 Token inválido. Navbar "Ingresar"
6	Borrá el <code>tieneRol('ADMIN')</code> && del link, entrá como CLIENTE	Ves el link. Click → /sin-permiso. Esconder ≠ proteger
7	Sacá /libros/nuevo del PrivateRoute, entrá como CLIENTE, llená el form	403 No tenés permiso... Volviste a C19. El back sigue ahí
8	En consola: <code>JSON.parse(atob(localStorage.token.split('.')[1]))</code>	Anda... hasta que un payload trae - o _: InvalidCharacterError. base64url ≠ base64
9	Back: <code>env.ts</code> "2h" → "30s", docker compose restart api, logueate, esperá, hacé un POST	401 Token expirado. La navbar pasa a "Ingresar" sola
10	Network al recargar logueado	Dos GET /auth/yo. Sacá <code><StrictMode></code> de main.tsx → uno. No es un bug: React verifica que el efecto sea idempotente



¿TERMINASTE? EXTENSIONES

#	Extensión	Pista
1	Volver a la página de origen después del login	<code>PrivateRoute: <Navigate to="/login" replace state={{ desde: useLocation().pathname }} />. Login: navigate(location.state?.desde ?? '/catalogo', { replace: true })</code>
2	Login.tsx con react-bootstrap	Hoy es HTML con estilos inline. Pasalo a <code><Form> + Form.Control + isValid</code> , como <code>LibroNuevo</code>
3	Menú desplegable con el nombre	<code>NavDropdown title={`Hola, \${usuario.nombre}`} con "Salir" adentro</code>
4	Botones editar / borrar en LibroCard solo ADMIN	<code>tieneRol('ADMIN') &&</code> . El DELETE ya existe en el back
5	Permisos por dueño (concepto)	Un CLIENTE edita su pedido: en la UI, <code>usuario.id === pedido.clienteId</code> . La regla de verdad va en el back (<code>req.usuario.id</code>). Sin implementar en la Librería
6	ApiError en las páginas	En LibroNuevo, <code>catch (e) { if (e instanceof ApiError && e.status === 403) ... }</code> . Distinguí "sin permiso" de "error del server"



LO QUE FALTA, FALTA A PROPÓSITO

La UI sabe quién sos y qué podés hacer. Lo que **no** hicimos hoy:

Pendiente	Qué es	Por qué hoy no
Implementar cookies httpOnly	El token fuera del alcance del JS	Se decidió y se justificó. Implementarlo toca CORS y el back. Alternativa de producción
Refresh tokens, "recordarme", 2FA	Sesiones largas sin token largo	Fuera del alcance de la materia
Permisos por dueño	Un CLIENTE edita lo suyo	Concepto en una línea (extensión 5). La lógica vive en el back
Roles dinámicos desde la DB	Permisos configurables sin deploy	Fuera del alcance de la materia
Redux / Zustand / React Query	Otro manejo de estado	No están en el programa. Para la sesión, context alcanza
Testing de rutas protegidas	¿Cómo se prueba PrivateRoute?	Clase de Testing (C21)
HTTPS y dominio real	localhost deja de existir	Clase de Deploy (C23)



LO QUE ENTRA EN EL EXAMEN DE C20

Concepto	En una línea
Autenticación vs autorización	¿Quién sos? / ¿Qué podés? El back responde 401 / 403. La UI reacciona y anticipa
La UI no es seguridad	Todo el front es editable. Esconder es UX; la regla vive en la API
Defensa en profundidad	Dos capas que dicen lo mismo. La de afuera no reemplaza la de adentro
Matriz de permisos + UI	Cada permiso: una implementación que vale (back) y una que se ve (UI)
AuthContext	La sesión como estado global: usuario, cargando, login, logout. Mismo patrón que C12
Rehidratación	F5 borra el estado, no el token. Se le pregunta al back (/auth/yo) quién sos
cargando	Tercer estado: "todavía no sé". Sin él, flash a /login
Ruta protegida	Layout route sin path + <Outlet />. Navigate replace para no atrapar al usuario
Por login vs por rol	Sin usuario → /login (401). Con usuario sin rol → /sin-permiso (403)
Renderizado condicional	{cond && <X />}. Esconder y proteger
Token vencido	401 con token → cerrar la sesión. El back avisa, el front obedece
localStorage vs httpOnly	XSS vs CSRF. La materia: localStorage con riesgo declarado. Producción: cookie

ACTIVIDADES

ACTIVIDAD INDIVIDUAL ENCUESTA

Instancia única:
20/09/2026

[AI-C20-A] Encuesta Bloque 3

Completar Encuesta de Cierre del Bloque 3:

Ingresar y completar la [encuesta](#) de satisfacción del Bloque 3, es anónima y rápida, nos permite medir y mejorar el dictado de la materia.

Hay tiempo hasta el Domingo 20 de septiembre, luego se cierra automáticamente.

Comunicar la finalización en Discord en un mensaje dentro del canal #actividades → hilo **[AI-C20-A] Encuesta Bloque 3**

ACTIVIDAD INDIVIDUAL

1° instancia hasta el
27/09/2026

2° instancia: **15/11/2026**
(Cierre Bloque 3)

[AI-C20-B] Autorización en la Librería

Referencia en el repo de ejemplo [branch C20](#)

En `ds2026-actividades`: copiar el sitio de la librería de la Clase 20 en **`ejercicios/c20-autorizacion/`**

1. `src/context/AuthContext.tsx` con el patrón de C12: `usuario`, `cargando`, `login`, `logout`, `estaAutenticado`, `tieneRol`. Hook `useAuth()` que tira si falta el Provider
2. Rehidratación al recargar con `GET /auth/yo`. El token **no** se decodifica en el front
3. `Login.tsx` usa `login()` del context. Ningún componente importa `sesion.ts` (solo `api.ts` y el context)
4. Navbar: "Ingresar" / "Hola, {nombre} · Salir" según la sesión
5. `src/components/PrivateRoute.tsx` como layout route con `<Outlet />`, `Navigate replace`, y `Spinner` mientras `cargando`. Con `rol` opcional
6. Página `/sin-permiso`. `/libros/nuevo` solo ADMIN: sin login → `/login`, CLIENTE → `/sin-permiso`, ADMIN entra
7. Link "Nuevo libro" visible solo para ADMIN
8. `apiFetch` tira `ApiError` con `status`. Un 401 con token cierra la sesión (evento + listener en el Provider)
9. `npx tsc -p tsconfig.app.json --noEmit` sin errores. Cero dependencias nuevas
10. Commiteá y avisá en `#actividades`
→ hilo **[AI-C20-B] Autorización en la Librería**

ACTIVIDAD GRUPAL

Hasta el 27/09/2026
(inclusive)

[AG-C20] Autorización en el TP

1. **Matriz de permisos del TP extendida** con la columna "Qué hace la UI", en Documentación/. Con los roles de **su** dominio
2. `AuthContext + useAuth()` con rehidratación contra su endpoint de "quién soy"
3. Rutas protegidas según su matriz: por login y por rol. Página "sin permiso" con su estilo
4. Renderizado condicional: navbar según sesión; botones y links según rol. Al menos una operación de escritura con su link/botón escondido **y** su ruta protegida **y** su 403 en el back
5. Cierre de sesión ante token vencido
6. `.env.example` del back con `JWT_SECRET=cambiam` y el vencimiento del token documentado
7. Repartir el trabajo: al menos **un PR con review cruzado por integrante**
8. Tarjetas de Trello movidas a "Done", retrospectiva del sprint y planning del siguiente
9. Aviso en `#actividades`
→ hilo **[AG-C20] Autorización en el TP**



ANTES DE LA PRÓXIMA CLASE

⚠ INSTALAR ANTES DE C21

El aula no tiene internet confiable.

Sin instalación = sin práctica.

```
bash

cd frontend
npm install -D vitest jsdom @testing-library/react
@testing-library/jest-dom

cd ../backend
npm install -D vitest supertest @types/supertest

npm ls vitest      # tiene que mostrar una versión
```

✓ Dudas → canal #consultas de Discord

 C21: Testing (28/09/2026)